Crypto Analyst

A verifiable, real-time crypto market-analysis engine

This document explains what the engine does, the principles it is built on, and how the codebase is organised. It is written for a software engineer who has not seen the project before. The system is the “analyst” half of a larger plan: a trustworthy market-analysis instrument that a trading bot will later consult before making decisions.

Project	Crypto Analyst (standalone analytics engine)
Backend	Python 3.11 - FastAPI, DuckDB, NumPy/Pandas, scikit-learn, TA-Lib
Frontend	React 18 + TypeScript (Vite), lightweight-charts
Data sources	Binance (public), Crypto.com, Coin Metrics, Deribit, sentiment engine
Size	~5,200 lines of Python across 45 modules; 20 React/TS files; ~109 tests
License of trust	Causal, calibrated, deterministic - no LLM, no look-ahead

Prepared as an architectural walkthrough · generated from the live codebase.

1 · Executive summary

Crypto Analyst is a market-analysis engine that turns raw exchange data into an honest, explainable read on a crypto asset. It ingests live price and derivatives data, computes a full suite of technical indicators, classifies the current market "regime" (trend / range / breakout / volatility shock), and layers on order-book microstructure, crowd positioning, on-chain activity, social sentiment and options-implied volatility. It then writes a plain-English analyst note and a per-metric breakdown, each grounded in the actual numbers.

The guiding idea is that **trust is a subsystem, not an afterthought.** Every datapoint carries its source and ingest time; every indicator is verified against the industry-standard TA-Lib library; every directional call is *measured* against history (does it actually beat a coin-flip?) rather than asserted. The engine is deliberately deterministic — there is no large-language-model in the analysis path — so the same inputs always produce the same, auditable output.

A React dashboard sits on top, but the engine is API-first: a FastAPI service exposes everything as JSON, so the future trading bot (or any other client) can consume the same reads the dashboard shows.

What problem it solves

- **Signal without snake-oil.** Most retail tools assert confidence they cannot back up. This engine reports the *measured* hit-rate and "lift" of each signal, and says so plainly when an edge is weak.
- **One source of truth.** All technical analysis is computed once, in Python, and verified — the frontend never re-implements indicators, so the chart and the commentary can never disagree.
- **Confluence over single signals.** The honest finding from the project's own backtesting is that no single short-horizon signal is reliable; the value is in *agreement* across regime, higher timeframe and crowd positioning, which is measurably better than any part alone.

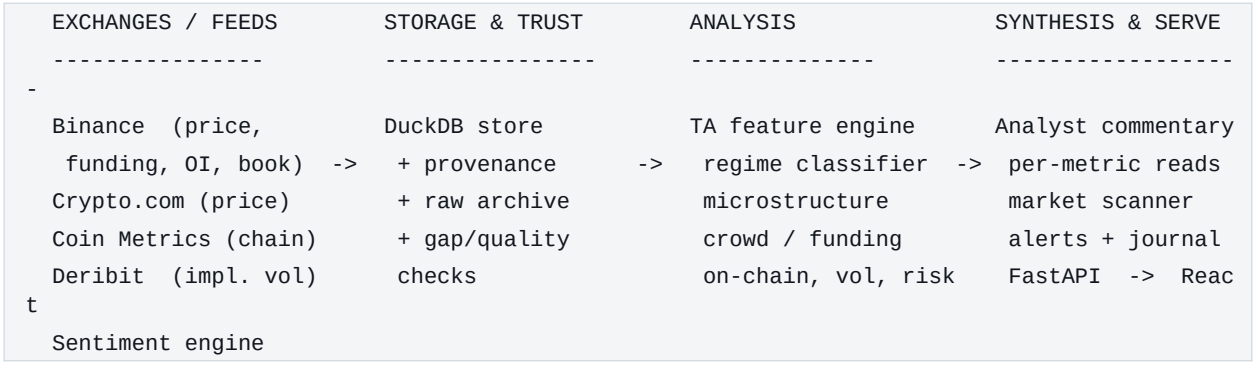
2 · Design philosophy

These five principles are visible throughout the code and are the reason the project exists in this form.

Principle	What it means in practice
Trust is a subsystem	Raw payloads are archived for replay; every row in the database records where it came from and when it was ingested; data-quality checks (e.g. gap detection) gate what the engine is willing to believe.
Causal / no look-ahead	The value of any indicator at bar *i* uses only data up to bar *i*. This is enforced by a dedicated test suite, so backtests cannot accidentally "see the future".
Calibrated, not asserted	Regime and signal calls are backtested against forward outcomes to produce a hit-rate, a base rate, and a lift (hit / base). The UI shows the lift, so weak edges are exposed rather than hidden.
Honest framing	Outputs are written as probabilistic reads with explicit invalidation levels and caveats — never a guaranteed forecast. The engine is candid that short-horizon direction is hard.
Deterministic & explainable	No LLM in the analysis path. Every sentence in the commentary is composed from computed values, so any claim can be traced back to a number.

3 · What the engine does, end to end

The pipeline runs left to right: raw data is ingested and trusted, turned into features, interpreted into signals, and finally synthesised into human-readable analysis served over an API.



3.1 Data ingestion & provenance

Adapters fetch public data from each venue behind a common interface, normalising every exchange's quirks into one internal symbol convention (`BTC_USDT`). Raw API payloads are archived to disk so any computation can be replayed later. Each stored candle records its source and ingest timestamp.

3.2 Storage & data quality

A single DuckDB database holds OHLCV candles, an ingest log, microstructure snapshots, funding history and data-quality results. A gap-detector flags missing bars before they can corrupt an indicator. (DuckDB is single-writer, so the read API opens the file read-only and the alert journal uses a separate database file to avoid lock contention.)

3.3 Technical-analysis engine

Computes ATR, EMA, SMA, RSI, MACD, Bollinger Bands, VWAP, ADX/DI and realized volatility. EMA is SMA-seeded; RSI/ATR/ADX use Wilder smoothing; Bollinger uses population std — all matching the TA-Lib reference exactly. Parity is checked in tests against TA-Lib, golden vectors, and mathematical invariants, and causality is checked by the no-look-ahead suite.

3.4 Regime classifier & calibration

A transparent, rule-based classifier labels the current tape as trend-up, trend-down, range, breakout, low-volatility drift, or flash-risk, with a confidence score and the human-readable reasons behind the call. A calibration harness walks history forward to measure how often each label actually resolved as expected (hit-rate vs base rate vs lift). Notably, this simple rule classifier *beat* a machine-learning benchmark out-of-sample — so it is the one in production.

3.5 Microstructure, crowd & derivatives

From the live order book and perpetual-futures endpoints the engine reads bid-ask spread (liquidity), depth imbalance (near-term order-flow pressure), funding rate and open interest (crowd leverage), and perp-vs-spot basis. Crowded longs are treated as a contrarian-bearish setup and crowded shorts as contrarian-bullish — with the contrarian edge *measured*, not assumed.

3.6 On-chain, sentiment & implied volatility

On-chain network activity (active addresses, transaction counts) is pulled from Coin Metrics' free community API and read for divergences against price. A separate real-time sentiment engine contributes Fear & Greed, social tone, a learned 6-hour up-probability, and positioning. Deribit's DVOL gives options-implied volatility, compared to realized vol to flag when the market is pricing in fear or complacency.

3.7 Levels, risk framing, scanner & alerts

Structural support/resistance levels are derived from swing pivots, range edges, volume point-of-control and round numbers. A risk module turns realized volatility into an expected 1-standard-deviation move, a price band, and reward-to-level ratios. A market scanner ranks the top coins for breadth and rotation, and a confluence-ranked alert system fires (and journals) setups so their real-world track record accrues over time.

3.8 Synthesis: the analyst note and per-metric breakdown

Finally, two deterministic writers turn all of the above into words. The **analyst note** fuses everything into one narrative — what's happening, what it means, and a calibrated outlook with an invalidation level. The **per-metric breakdown** does the opposite: a focused read for each single signal (regime, RSI, MACD, ADX, funding, microstructure, positioning, market context, on-chain), each giving the reading, its meaning, and where it might go based on that metric alone.

4 · Architecture & request flow

The system is API-first. The Python service is the single source of truth for all analysis; the React app is a pure presentation layer that never recomputes a number. During development, Vite proxies `/api` to the FastAPI service to sidestep browser CORS and IPv4/IPv6 issues.

Layer	Responsibility	Key tech
Collectors / CLIs	Pull and persist candles, funding, microstructure on demand or on a schedule.	analyst.ingest, collect_*
Storage	Durable OHLCV + provenance + quality + journal.	DuckDB
Analysis core	Indicators, regime, crowd, on-chain, vol, levels, risk, scanner, alerts.	NumPy / Pandas / scikit-learn
Synthesis	Compose the analyst note and per-metric reads from computed values.	commentary, metric_reads
API	Expose everything as JSON; cache live fetches; serve the dashboard's data.	FastAPI / uvicorn
Frontend	Charts, cards and the Simulate-driven commentary; presentation only.	React + lightweight-charts

A typical "Simulate" request: the dashboard calls `/commentary/BTC_USDT`; the API loads fresh candles (auto-refreshing stale data), gathers microstructure, sentiment, funding, positioning, higher-timeframe context, on-chain and implied vol, runs the calibration, then asks the two writers to produce the note plus the per-metric breakdown — returned as one JSON document the React app renders.

5 · Code organisation — backend

The Python package `analyst/` is organised by responsibility. Sizes are approximate line counts, included to show where the weight sits.

5.1 Package layout

```
analyst/
  config/         settings: paths, exchange selection, timeframe tables
  ingestion/      data-source adapters + raw archival
  storage/        DuckDB store + the separate alert journal
  quality/        data-quality checks (gap detection)
  features/       indicator math + the snapshot/feature engine
  regime/         classifier, calibration, ML benchmark, tuning
  microstructure/  order-book snapshot compute + collector
  api/            FastAPI app (all HTTP endpoints)
  *.py            analysis + synthesis modules (see tables below)
tests/            ~109 tests: parity, no-look-ahead, calibration, quality
frontend/         React + TypeScript dashboard
scripts/          helper scripts;  run-*.bat one-click launchers
```

5.2 Core data & trust layer

Module	Purpose	~LOC
ingestion/base.py	The data-source contract (interface every venue adapter implements).	—
ingestion/binance.py	Binance public adapter: klines, funding, open interest, book, mark/index.	259
ingestion/cryptocom.py	Crypto.com adapter with pagination (300-candle cap workaround).	254
ingestion/coinmetrics.py	Coin Metrics community API (on-chain activity, keyless).	—
ingestion/deribit.py	Deribit DVOL implied-volatility source (BTC/ETH).	—
ingestion/sentiment.py	Client for the external real-time sentiment engine.	82
ingestion/archive.py	Raw-payload archival for replay.	—
ingestion/factory.py	Selects the active exchange adapter from config.	—
storage/db.py	DuckDB store: OHLCV, ingest log, micro, funding, quality, provenance.	289
storage/journal.py	Thread-safe alert journal (separate DB file).	—
quality/gaps.py	Missing-bar / gap detection — the first trust check.	—
models.py	Pydantic data models (Candle, snapshots, etc.).	87
ingest.py	Ingestion orchestrator + command-line entry point.	101

5.3 Analysis core

Module	Purpose	~LOC
features/indicators.py	ATR/EMA/SMA/RSI/MACD/Bollinger/VWAP/ADX/realized-vol math (TA-Lib parity).	211
features/engine.py	Builds the latest-value indicator snapshot used everywhere.	120
regime/classifier.py	Transparent rule-based regime labels + confidence + reasons.	176
regime/calibration.py	Walk-forward hit-rate / base-rate / lift for each regime.	128
regime/model.py	Trees-first ML benchmark (kept for comparison; rules win OOS).	197
regime/tune.py	Walk-forward threshold tuning.	113
regime/derivatives.py	Derivative regime features.	—
microstructure/compute.py	Computes spread / depth / imbalance / basis from a live book.	—
microstructure/collect.py	Periodic microstructure snapshot collector.	85
crowd.py	Funding-crowding contrarian signal + its measured calibration.	75
onchain.py	On-chain activity snapshot + price-divergence read.	—
vol.py	Implied (DVOL) vs realized volatility context.	—
levels.py	Structural support/resistance (pivots, POC, round numbers).	138
risk.py	Expected move, price band, reward-to-level (pure vol math).	75
scanner.py	Multi-coin breadth/rotation scan + standout setups.	124
alerts.py	Confluence-ranked alert detection.	87
alert_calibration.py	Per-alert-type forward hit-rate / lift.	116
outlook_calibration.py	Confluence calibration (regime + crowd + higher timeframe).	113

5.4 Synthesis & service

Module	Purpose	~LOC
commentary.py	The analyst note: what's happening, what it means, calibrated outlook.	512
metric_reads.py	Focused per-metric reads (reading / meaning / outlook + bias).	368
api/app.py	FastAPI app: every HTTP endpoint, caching, live-fetch helpers.	657

6 · The API surface

All analysis is reachable as JSON. The dashboard uses these; so will the trading bot.

Endpoint	Returns
/health	Store reachability + per-symbol data freshness.
/provenance/{sym}	Where each series came from and when it was ingested.
/candles/{sym} · /chart/{sym}	Raw candles, or candles + full indicator series for charting.
/indicators/{sym}	Latest technical-analysis snapshot.
/regime/{sym} (+ /calibration)	Current regime call + confidence + reasons; measured hit-rate/lift.

Endpoint	Returns
/microstructure/{sym}	Live (or recent) spread / depth / OI / funding / basis.
/commentary/{sym}	The analyst note **plus** the per-metric breakdown.
/scan · /alerts · /alert-journal	Market breadth scan, live alerts, fired-alert track record.
/sentiment/{sym} · /onchain/{sym} · /vol/{sym}	Sentiment bundle, on-chain activity, implied-vol context.
/quality/{sym}	Recent data-quality (gap) results.

7 · Code organisation — frontend

The dashboard is a small React + TypeScript app (Vite). It is presentation-only: it fetches JSON and renders it. Charts use lightweight-charts, built once and updated in place so live refreshes don't flicker. The layout leads with the synthesised analyst read, then breaks the evidence apart into collapsible sections.

File	Role
App.tsx	Top-level state, data fetching, layout, and the Simulate flow.
api.ts	Typed fetch helpers with retry; talks to the /api proxy.
types.ts	TypeScript interfaces mirroring the API's JSON shapes.
components/Dashboard.tsx	Six time-synced chart panes (price+levels, RSI, MACD, ADX, funding, regime).
components/CommentaryCard.tsx	The analyst note + the Simulate button.
components/MetricBreakdown.tsx	The per-metric analyst reads grid.
components/RegimeCard.tsx	Regime label, confidence, reasons, calibration.
components/MicrostructureCard.tsx	Live order-book snapshot (with freshness/live tag).
components/SentimentCard.tsx	Sentiment & context, grouped with a synthesized read + tooltips.
components/OnChainCard.tsx	On-chain network-activity read.
components/MarketScanCard.tsx	Top-coin breadth scan table.
components/AlertsCard.tsx / AlertJournalCard.tsx	Live alerts and their accruing track record.
components/TrustBar.tsx	Provenance / freshness strip (the 'trust' header).
components/Section.tsx / Controls.tsx / ErrorBoundary.tsx	Collapsible section, pickers, per-card error isolation.

8 · Tech stack, testing & how to run

8.1 Dependencies

Area	Libraries
Backend core	FastAPI, uvicorn, pydantic, httpx, tenacity (retries)
Data / math	DuckDB, NumPy 2, Pandas, scikit-learn
Reference / test	TA-Lib (gold-standard indicator parity), pytest
Frontend	React 18, TypeScript, Vite, lightweight-charts v4

8.2 Testing & verification (~109 tests)

- **Indicator parity** — every indicator checked against TA-Lib, golden vectors and invariants.
- **No look-ahead** — dedicated suites prove indicators and the regime classifier are causal.
- **Calibration** — regime, alert and confluence calibrations are tested for correctness.
- **Data & pipeline** — storage, ingestion, pagination, gaps, microstructure, sentiment, on-chain.

8.3 Running it

```
# one-click (Windows): frees ports, ingests, starts API + dashboard
run-analyst.bat            # analyst only
run-all.bat                # analyst + sentiment engine

# manual
uv venv --python 3.11 .venv
uv pip install -e .
python -m analyst.ingest --symbols BTC_USDT,ETH_USDT --timeframe 1h --count 300
python -m uvicorn analyst.api.app:app --reload
python -m pytest
```

8.4 Status

Foundation, the TA engine, microstructure, the regime classifier + calibration, and the accuracy study are complete; dashboards, alerts, on-chain and sentiment/options integration are built and in active refinement. The next phase — once the analyst is fully trusted — is the grid trading bot that will consult this engine. The honest headline from the research so far: *single short-horizon signals sit near the predictability floor, but measured confluence beats the parts.*

Crypto Analyst · architectural overview · all figures generated from the live codebase.